Regex drill: "give a regular expression for this"

Eight questions in the mock's format (alphabet, description, must-match and must-not examples), each with the parts cut, the expression, the checks and the trap

Compilers

2026-11-02

What the mock asked. Mock 1 had two regex questions: 1.7(b) *numeric literals* ("give a regular expression; say which of these match and why the others do not") and 2.3(a) *comments* over a reduced alphabet $\{/, *, x\}$, followed by its DFA and the question whether nested comments are regular. Mock 2 tested the scanner (longest match with backup), not regex writing. No question asked for a JFlex/flex specification: flex appears only in the answers, as the behaviour behind rule order (longest match, then the first rule) and nested comments (a start condition plus a depth counter).

The method every time (§2 of the methods sheet). Cut the lexeme into parts; mark each part mandatory, optional `?`, or repeated `*`/`+`; write one sub-expression per part with | inside a part; then split every must-match example into the parts and, for every must-not example, name the part that rejects it. D stands for `[0-9]`, L for `[a-zA-Z]`. Write x for "any other character" when the question reduces the alphabet.

1 Integers with digit separators

Practice

Given. Alphabet $D \cup \{_\}$. An integer literal is one or more digits, where groups of digits may be separated by single underscores, as in Java/Python.

Your task. (a) Write the regex. (b) Show the matches of `7`, `1_000`, `12_34_5`. (c) Explain why `_1`, `1_`, `1__0` are not matched.

Solution

Parts. A first group D^+ (mandatory), then zero or more of "underscore then group".

$$D^+ \, (\, _ \, D^+ \,)^*$$

Checks. `7`: one group. `1_000`: group `1`, then `_000`. `12_34_5`: group `12`, then `_34`, then `_5`. `_1`: the first part is D^+, mandatory, and `_` is not a digit. `1_`: the repeated part is $_D^+$, so an underscore must be followed by a digit. `1__0`: same part; after the first `_` comes another `_`, not a digit.

Gotcha

$(D \mid _)^+$ or $D^+(_^*D^+)^*$: the first accepts `__`, the second accepts `1__0`. "Single underscore between digits" means the underscore lives *inside* the repeated part, never at either end.

2 Floating-point literals, the other convention

Practice

Given. Alphabet $D \cup \{.\}$. A float has an optional integer part and an optional fraction part, but **at least one digit somewhere**: `3.`, `.5`, `3.5` and `3` are all floats (unlike mock 1.7(b), where the integer part was mandatory).

Your task. (a) Write the regex. (b) Show the matches of `3`, `3.`, `.5`, `3.5`. (c) Explain why `.`, `3..5`, `3.5.` are not matched.

Solution

Parts. Two shapes, so two alternatives: either the integer part is present (then the fraction is optional and may be empty after the dot), or it is absent (then the dot and at least one digit are mandatory).

$$D^+ (\, . \, D^*)? \quad | \quad . \, D^+$$

Checks. `3`: left alternative, no dot. `3.`: left, dot with $D^* = \varepsilon$. `.5`: right alternative. `3.5`: left. `.`: the right alternative needs D^+ after the dot, the left needs D^+ before. `3..5`: after `3.` the regex allows only digits; a second dot has no part to go to. `3.5.`: same, the dot appears only once in either alternative.

Gotcha

$D^*.?D^*$ matches the empty string and the lone dot `.`: "at least one digit" is the whole difficulty and must be enforced by the shape of the alternatives, not left to D^*.

3 Identifiers with at least one letter

Practice

Given. Alphabet $\{a, _, 1\}$ (a stands for any letter). An identifier starts with a letter or an underscore, continues with letters, underscores and digits, and must contain **at least one letter** (so `_` and `__1` are not identifiers).

Your task. (a) Write the regex. (b) Show the matches of `a`, `_a1`, `__a`. (c) Explain why `1a`, `_`, `_1` are not matched.

Solution

Parts. Everything before the first letter can only be underscores (no digit may come before the first letter, since the first symbol is a letter or an underscore and an underscore run cannot contain a digit). Then the first letter, mandatory. Then anything.

$$\texttt{_}^{*}\ a\ (\,a \mid \texttt{_} \mid 1\,)^{*}$$

Checks. `a`: zero underscores, the letter, nothing. `_a1`: one underscore, letter, then `1`. `__a`: two underscores, letter.
`1a`: the prefix part is $\texttt{_}^{*}$, a digit cannot precede the first letter. `_`: the mandatory letter is missing. `_1`: after the underscores a letter must come, not a digit.

Gotcha

"At least one letter" is a *global* constraint, and regexes have no "somewhere": you encode it by anchoring on the **first** letter and describing what may come before it. The same move gives "contains `ab`": $(a \mid b)^{*}\, ab\, (a \mid b)^{*}$.

4 SQL string literals with doubled quotes

Practice

Given. Alphabet $\{\texttt{'}, x\}$. A string literal is enclosed in single quotes; inside, a quote is written as two consecutive quotes (`'it"s'` is the string `it's`). There is no backslash.

Your task. (a) Write the regex. (b) Show the matches of `"`, `'x'`, `'it"s'` (letters are x). (c) Explain why `'it's'` and `"'` are not matched as one literal, and what a longest-match scanner returns on each.

Solution

Parts. Opening quote; body items, each either x or the pair `"`; closing quote.

$$\texttt{'}\ (\,x\ \mid\ \texttt{"}\,)^{*}\ \texttt{'}$$

Checks. `"`: zero items. `'x'`: one item. `'it"s'`: items x, x, the pair `"`, x, then the closing quote. `'it's'`: after `'it` the next symbol is `'` followed by s, which is not the pair, so that quote closes the literal at `'it'`; the leftover `s'` is not part of it. A longest-match scanner returns the token `'it'`, then scans `s` and `'` separately (the final lone quote is a lexical error or the start of a new literal). `"'`: after the opening quote, `"` is one body item, and then the closing quote is missing; the scanner backs up to the last accepting position, which is after `"`, and returns the empty literal `"`, leaving one `'`.

Gotcha

`'`$(x \mid$ `'`$)^{*}$`'` ("anything between quotes") accepts `'it's'` and, worse, with longest match it swallows two literals and the code between them into one token: the mock's 2.3(a) trap in a different costume.

5 Clock times

Practice

Given. Alphabet $D \cup \{:\}$. A time is `HH:MM` on a 24-hour clock, always with two digits per field: `00:00` to `23:59`.

Your task. (a) Write the regex. (b) Show the matches of `09:30`, `23:59`. (c) Explain why `24:00`, `9:30`, `12:60` are not matched.

Solution

Parts. The hour has two cases, split on the first digit: first digit `0` or `1` with any second digit, or first digit `2` with second digit `0`–`3`. The minute is `0`–`5` then any digit.

$$([01]\, D \;\mid\; 2\, [0\text{-}3])\; :\; [0\text{-}5]\, D$$

Checks. `09:30`: hour case 1 (`0`,`9`), minutes `3`,`0`. `23:59`: hour case 2 (`2`,`3`), minutes `5`,`9`. `24:00`: first digit `2` forces the second into `[0-3]`. `9:30`: the hour needs two digits, no alternative has one. `12:60`: the minute's first digit is limited to `[0-5]`.

Gotcha

$D\,D{:}D\,D$ "and check the range later" is a semantic check, not a regex; if the question says *regular expression for valid times*, the ranges go into the alternatives. A numeric range is always written by **splitting on the leading digit**.

6 No two consecutive `b`'s

Practice

Given. Alphabet $\{a, b\}$. The language of all strings (including ε) that do not contain `bb`.

Your task. (a) Write the regex. (b) Show how ε, `b`, `abab`, `bab` match. (c) Explain why `bb` and `abba` do not. (d) Give the three-state DFA.

Solution

Parts. Every b except possibly the last must be followed by an a, so the string is a sequence of blocks a or ba, optionally ending with a lone b.

$$(a \;\mid\; ba)^*\; (b)?$$

Checks. ε: zero blocks, no final b. `b`: zero blocks, the final b. `abab`: blocks a, ba, then the final b. `bab`: block ba, final b.
`bb`: the first b must start a block ba (no a follows) or be the final b (but another symbol follows). `abba`: after block a, the first b needs an a next; it gets a b.

DFA. States: 0 "last symbol not b" (start, accepting), 1 "last symbol b" (accepting), dead. $0 \xrightarrow{a} 0$, $0 \xrightarrow{b} 1$, $1 \xrightarrow{a} 0$, $1 \xrightarrow{b}$dead. Three states, as the distinguishable strings ε, b, bb demand.

Gotcha

$(a \mid ba)^*$ alone loses every string ending in b (`b`, `ab`); $(a \mid b)^*$ with "but not `bb`" written in words is not a regex. "Does not contain w" is written by describing what the string *is* made of, never by negation.

7 Comments, a different pair of delimiters

Practice

Given. Alphabet $\{(,), *, x\}$. A Pascal comment opens with `(*`, closes with the **first** `*)`, and is not nested.

Your task. (a) Write the regex (the mock's 2.3(a) with new symbols: do it without looking). (b) Show how `(**)`, `(*x*)`, `(*(*x**)` match. (c) Explain why `(*x*)x*)` is not one comment. (d) Is the nested version regular?

Solution

Parts. Opener `(*`; a body that never contains `*)`: items x, `(`, a lone `)` (harmless: only `*)` closes), or a run of stars followed by something that cannot close, i.e. x or `(`; the closer is a run of one or more stars and `)`.

$$(* \ (\ x \ \mid \ (\ \mid \ ) \ \mid \ *^+ (x \mid () \ )^* \ *^+ \ )$$

Checks. `(**)`: opener, empty body, closer `*)`. `(*x*)`: body x, closer. `(*(*x**)`: body `(`, then the item $*x$ (a star run followed by x), closer `**)` (the star run of the closer absorbs both stars). `(*x*)x*)`: after `(*x` the symbols `*)` can only be the closer (a star run inside the body must be followed by x or `(`), so the comment ends at `(*x*)`; the rest is outside it.

Nested. No: accepting `(* a (* b *) c *)` as one comment needs the number of open `(*`, and the strings $((*)^1, ((*)^2, \ldots$ are pairwise distinguishable (by how many `*)` close them), so no finite DFA works. A lexer uses a start condition and a depth counter.

Gotcha

Forgetting that the body may contain a star or a lone `)`: `(*`$(x \mid ()^*$`*)` rejects `(*x**)`, `(*(*x**)` and `(*)*)`. The mock's alphabet has the same trap with a lone `/`: the second symbol of the closer is harmless on its own, only the pair closes. Forgetting that the closer may have several stars: `...*)` with a body item $*^+x$ still fails on `(***)`, which needs $*^+$`)` as the closer.

8 Identifiers that are not the keyword `if`

Practice

Given. Alphabet $\{i, f\}$. The language of nonempty strings that are not exactly `if`.

Your task. (a) Write the regex. (b) Explain why a real lexer never writes this regex, and how it gets the same effect.

Solution

Parts. Split by length. Length 1: $i \mid f$. Length 2, all but `if`: $ii \mid fi \mid ff$. Length ≥ 3: anything.

$$i \mid f \mid ii \mid fi \mid ff \mid (i \mid f)(i \mid f)(i \mid f)^+$$

Why nobody writes it. Regular languages are closed under complement, so the regex always exists, but it is unreadable and grows with every keyword. A lexer lists the keyword rule `if` *before* the identifier rule $(i \mid f)^+$: on `if` both rules match the same length and the tie goes to the first rule; on `iff` only the identifier rule gives the longest match. Mock 2.4 is this question with the two rules swapped (then `if` becomes an identifier and the keyword rule is unreachable).

Gotcha

Writing "$(i \mid f)^+$ except `if`" or $\neg$`if`: not regex syntax. If a question asks for the regex, build it by length or by first-difference; if it asks what the lexer does, answer with rule order and longest match.